

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
PROGRAMA DE PÓS-GRADUAÇÃO EM MICROELETRÔNICA

ULISSES MAFFAZIOLI
VICENTE VEIGA

Relatório 4: Trabalho Final

Projeto de um Sistema VLSI Digital (MIC61) /
Concepção de Circuitos Integrados II (MIC02)
Trabalho de Disciplina – Relatório 4

Orientador: Prof. Dr. Sergio Bampi

Porto Alegre
2026

Sumário

1 INTRODUÇÃO	3
2 SÍNTESE FÍSICA COM SKY 130	5
2.1 Preparação das Entradas	5
2.2 Floorplan e Planejamento de Alimentação	6
2.3 Placement e CTS	6
2.4 Roteamento	8
2.5 Verificação e Limitações	9
3 SÍNTESE FÍSICA COM SILTERRA 180	11
3.1 Sobre o fluxo	11
3.2 Diferenças com relação aos Trabalhos anteriores	11
3.3 Os resultados	13
3.4 Verificação e Signoff	15
4 CONCLUSÃO	16

1 INTRODUÇÃO

Este relatório documenta as etapas de **síntese física** e **sign-off** do processador RVX, dando continuidade ao fluxo de síntese lógica apresentado no relatório anterior. O trabalho foi conduzido em dois contextos tecnológicos distintos: o PDK comercial **Silterra 180 nm**, no qual o projeto foi levado até *tapeout*, e o PDK *open source* **SkyWater sky130**, no qual todas as etapas do fluxo foram executadas, porém sem que fosse possível concluir o sign-off.

Partiu-se do mesmo núcleo RVX, agora estendido com **ECC (Error-Correcting Code)** na memória e no banco de registradores, tornando-o tolerante a falhas. A partir desse novo RTL, sintetizou-se uma versão com 1 KB de memória RAM para o fluxo em Silterra 180. Na implementação em sky130, a memória foi mantida em apenas 128 bytes, a fim de reduzir a área e otimizar o tempo de processamento das etapas de síntese física.

A implementação em **Silterra 180** foi concluída com sucesso e encaminhada para *tapeout*. Realizamos a inserção do anel de PADs com os respectivos *filler pads*, efetuamos todas as conexões de alimentação e sinal, e resolvemos a totalidade das violações de DRC do *core*. O projeto passou nas verificações de **LVS** e **Formality**, e as simulações pós-síntese lógica e pós-síntese física apresentaram resultados funcionais corretos.

Já a implementação em **sky130** esbarrou em limitações da biblioteca de PADs do PDK disponível no ambiente CADMicro, que não se mostrou plenamente caracterizada para um fluxo completo de anel de PADs. Como o projeto é *core-limited*, o preenchimento do anel exige *filler pads*, que existem apenas na família *sky130_ef_io*, e não na família *sky130_fd_io*. Além disso, foi necessário aplicar manualmente correções (*patches*) ao arquivo LEF para viabilizar a conexão de VDD e GND ao *core ring*. Apenas essas conexões de alimentação puderam ser estabelecidas; as demais conexões dos PADs não puderam ser concluídas. Por essas limitações, e também por restrições de tempo, o fluxo em sky130 não avançou até o sign-off, embora todas as etapas do processo tenham sido percorridas.

Para esta síntese final, a síntese lógica foi refeita a fim de ajustar o período de *clock* para 15 ns, ante os 11 ns adotados no Relatório 2 e os 5,4 ns alcançados no primeiro relatório. O relaxamento progressivo da restrição temporal reflete a crescente complexidade do fluxo, com a inclusão da cadeia de varredura e da análise MMMC completa.

O restante deste relatório está organizado da seguinte forma: o Capítulo 3 detalha

a síntese física em Silterra 180, desde a inserção de PADs até o sign-off; e o Capítulo 2 relata o fluxo de síntese física em sky130, descrevendo os passos executados e as limitações encontradas nos PADs do PDK.

2 SÍNTESE FÍSICA COM SKY 130

Ulisses Maffazioli

2.1 Preparação das Entradas

O ponto de partida da síntese física é o conjunto de entradas gerado ao final da síntese lógica no Genus, complementado por dois arquivos produzidos manualmente: `rvx_top.v`, que descreve o *wrapper* do núcleo com o anel de PADs, e `rvx_top.io`, que define o posicionamento dos PADs ao longo das bordas do *die*.

O anel é composto por 33 PADs, sendo 17 destinados a sinais e 16 à alimentação. Optou-se por distribuir um par de PADs de alimentação de I/O (VDDR GNDR) e um par de alimentação do *core* (VDD GND) em cada um dos quatro lados do *die*, de modo a compensar a ausência de continuidade elétrica nos *corners* e garantir a distribuição uniforme da alimentação ao longo de todo o anel (ver Figura 2.1).

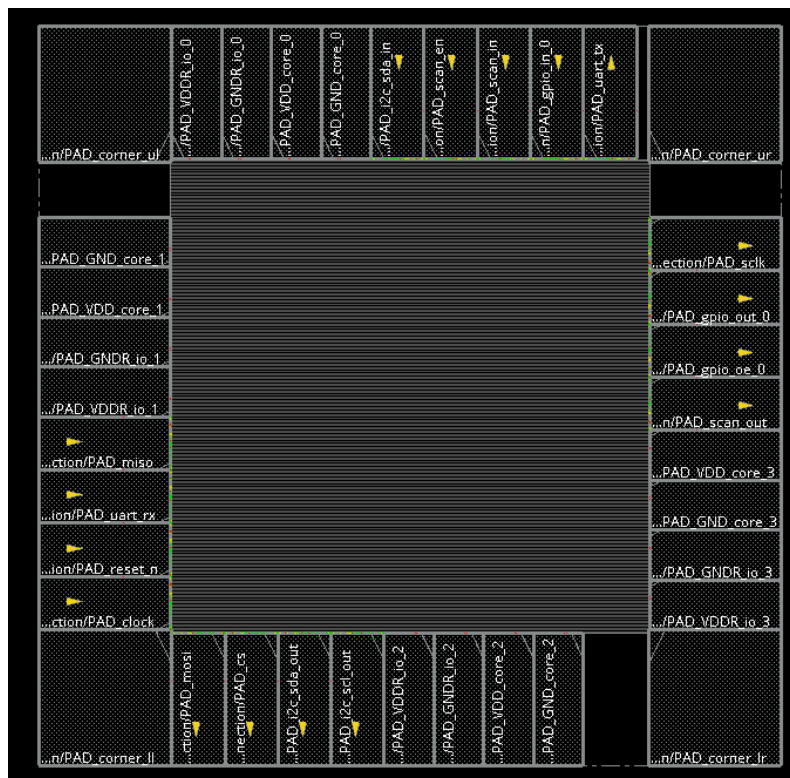


Figura 2.1 – Distribuição dos PADs

Antes de invocar o *setup* do Innovus, foi necessário um ajuste na cadeia de *scan*. O `write_scandef` do Genus gera o DEF referenciando apenas o módulo do núcleo (`rvx`), enquanto na síntese física o núcleo é instanciado dentro de `rvx_top`, junto ao anel de PADs. Foi criado, portanto, um alvo no *Makefile* (`fix_scandef`) que, via `sed`, atualiza

o cabeçalho DESIGN de *rvx* para *rvx_top* e prefixa todos os caminhos dos *flip-flops* da cadeia com o nome da instância do núcleo (*rvx/*). Sem essa correção, o Innovus não localiza os elementos da cadeia na nova hierarquia.

2.2 Floorplan e Planejamento de Alimentação

O *die* foi definido com dimensões inteiras de $1120 \times 1100 \mu m$ e margem de $30 \mu m$ entre núcleo e anel de PADs. A opção `-noSnapToGrid` foi necessária porque o menor *filler* disponível é de $1 \mu m$: folgas fracionárias menores seriam impreenchíveis e inviabilizariam o fechamento do anel. As folgas remanescentes foram preenchidas com *filler pads* da família *sky130_ef_io*.

A malha de alimentação (VDD/GND) foi construída em duas etapas. O *core ring* emprega segmentos horizontais em *met5* e verticais em *met4*, com largura de $6 \mu m$ e espaçamento de $4 \mu m$ entre as duas redes. Sobre ele, distribuiu-se uma malha de *stripes* de $6 \mu m$ de largura e $15 \mu m$ de espaçamento interno ao par: os verticais em *met4*, com passo de $120 \mu m$, e os horizontais em *met5*, com passo de $110 \mu m$. Em ambos os casos, o intervalo de camadas foi limitado entre *met2* e *met5*, tanto no anel núcleo–PAD quanto nos anéis de bloco (ver Figura 2.2).

A conexão física da alimentação foi realizada com `sroute`: os *trunks* ligam os PADs de VDD/GND ao *core ring*, enquanto os *rails* (*follow-pins* em *met1*) distribuem a alimentação ao longo das fileiras de *standard cells*, com faixa de vias entre *met1* e *met5* (ver Figura 2.3).

2.3 Placement e CTS

O posicionamento das células foi realizado com `place_design`, seguido do reordenamento da cadeia de *scan* (`scanReorder`) para reduzir o comprimento das interconexões e da atribuição dos pinos de I/O (`assignIoPins`).

Antes da síntese da árvore de *clock*, o *clock* foi redefinido sobre o pino *core-side* do PAD (`PAD_connection/PAD_clock/IN`), com período de $15 ns$, garantindo que apenas esse relógio permanecesse ativo. Definiu-se uma regra de roteamento não-padrão (`CTS_2W1S`, com largura e espaçamento dobrados em *met2*–*met3*) aplicada aos segmentos *trunk* e *top* da árvore, além dos conjuntos de *buffers* e inversores de *clock*

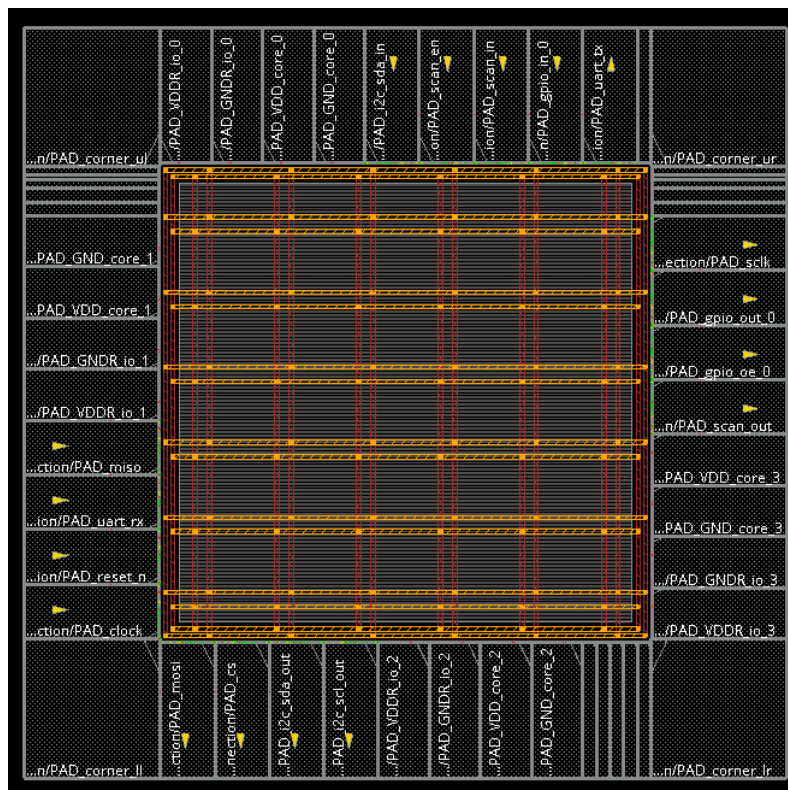


Figura 2.2 – CoreRing e Stripes

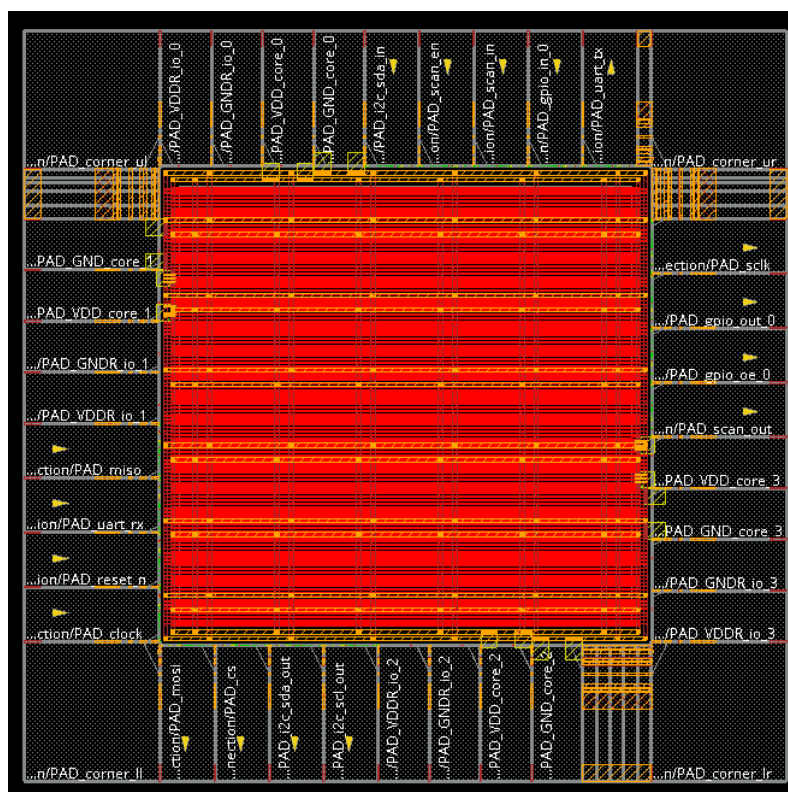


Figura 2.3 – Inserção de Rail e Trunks

da biblioteca `sky130_fd_sc_hd`. A árvore foi então sintetizada com `cconst_design`, precedida e sucedida pelas otimizações `optDesign -preCTS` e `optDesign -postCTS` (ver Figura 2.4).

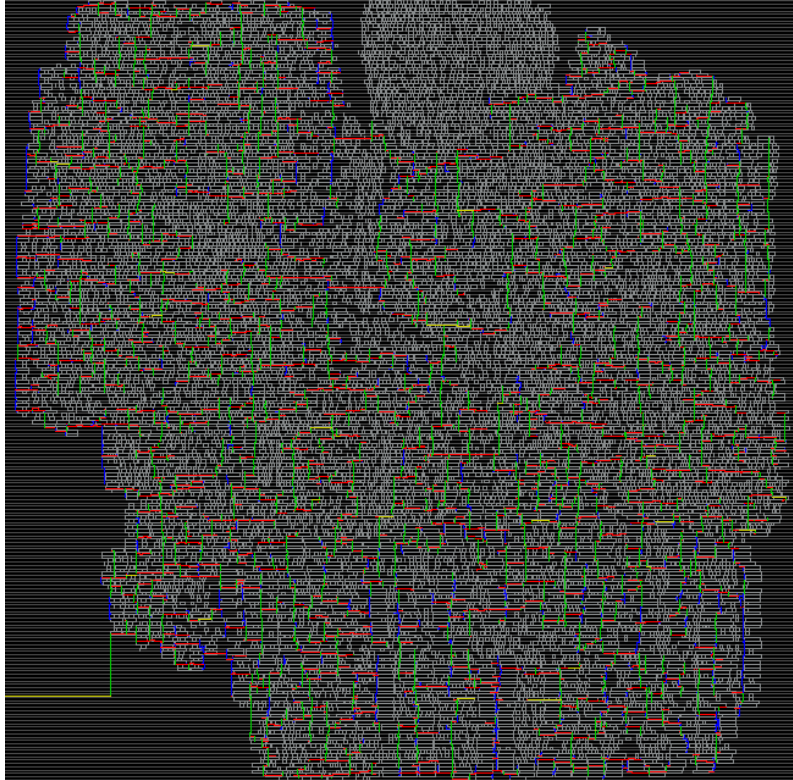


Figura 2.4 – Árvore de Clock

2.4 Roteamento

Previamente ao roteamento, a correção de antena foi habilitada via `setNanoRouteMode`, definindo-se a célula de diodo `sky130_fd_sc_hd__diode_2`, a inserção automática de diodos (`-route_antenna_diode_insertion true`) e a correção de antena no roteamento detalhado (`-route_detail_fix_antenna true`). A análise de *timing* foi configurada para o modo *on-chip variation* e o roteamento global e detalhado foi executado com `routeDesign -globalDetail`. Em seguida, aplicaram-se as otimizações pós-rotaemento (`optDesign -postRoute`), o roteamento incremental de correção (`ecoRoute -target`) e uma otimização final incremental (`optDesign -incr`).

A Figura 2.5 apresenta a cadeia de varredura (*scan chain*) já roteada sobre o *layout* final, com os *flip-flops* reordenados durante o *placement* interligados ao longo de todo o núcleo.

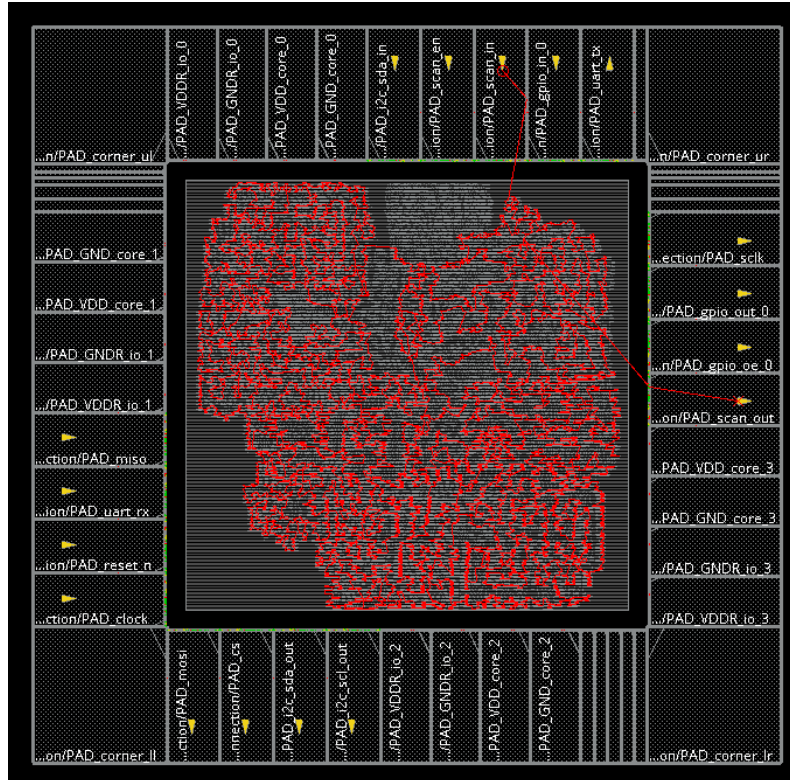


Figura 2.5 – Cadeia de varredura roteada sobre o *layout* final

2.5 Verificação e Limitações

A verificação de DRC do *core* foi conduzida com `verify_drc`. Excluindo-se as redes de alimentação (`-exclude_pg_net`), o roteamento de sinal apresentou **zero violações**, confirmando que as interconexões do núcleo foram concluídas de forma limpa. As violações remanescentes observadas na verificação completa concentraram-se exclusivamente na interface com a malha de alimentação e com o anel de PADs. A verificação de antena (`verify_antenna`), executada sobre a totalidade das redes, também não acusou violações.

Não foram realizados os testes de conectividade (`verifyConnectivity`), e três violações de DRC associadas aos PADs permaneceram sem resolução. Essas violações decorrem das limitações da biblioteca de PADs do PDK descritas na Seção ??, que impediram a conclusão plena das conexões do anel. Por essa razão, o fluxo em sky130 não avançou até o *sign-off*.

A Figura 2.6 apresenta a versão final do *chip*, com o núcleo roteado, a malha de alimentação e o anel de PADs posicionados ao longo das bordas do *die*.

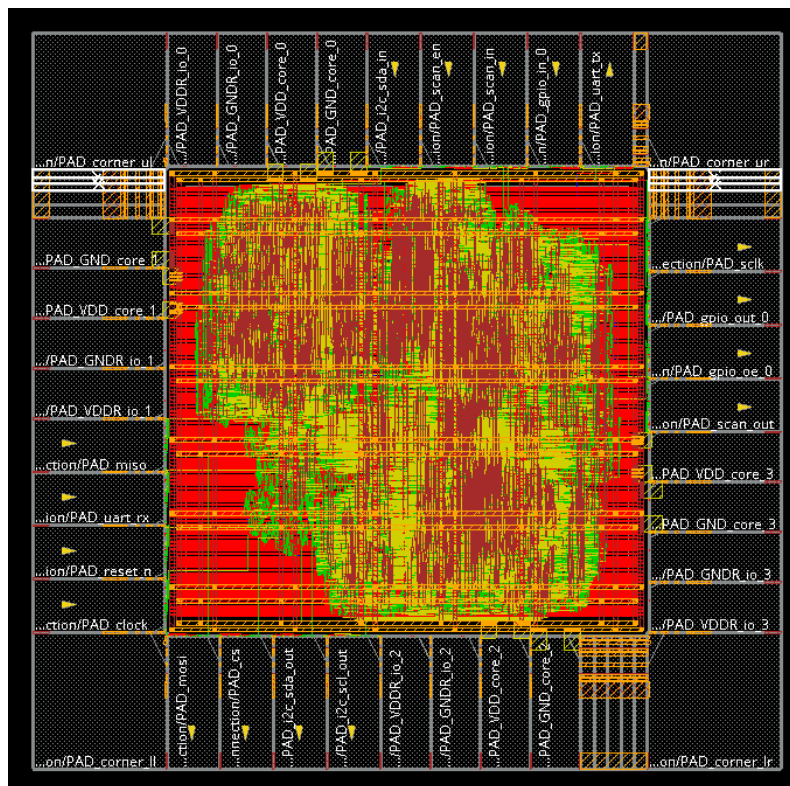


Figura 2.6 – Versão final do *chip*

3 SÍNTESE FÍSICA COM SILTERRA 180

Vicente Veiga

3.1 Sobre o fluxo

A síntese para o nó Silterra180 foi feita em razão de uma oportunidade de *tapeout* que nos foi cedida através de uma colaboração inesperada de nossa Prof^a. orientadora Fernanda Kastensmidt com a UPM (Universiti Putra Malaysia). Em razão da repentinidade dessa cessão, tivemos o que muitos chamariam de "pouquíssimo tempo" para a realização desse projeto, que foi entregue para *tapeout* no dia 30 de Junho de 2026, até o momento sem retorno da Fab, mas com aprovação do Broker CEDEC. Todo o processo foi realizado remotamente, numa máquina pertencente à UPM, onde o PDK estava instalado. O fluxo usou ferramentas Synopsys: DC Compiler, IC Compiler I, Formality e VCS. Ao final do processo, perdemos acesso à máquina e, embora tenhamos versionado os scripts de fluxo, não temos todos os logs de execução das ferramentas, somente uma parte deles, que foi incluída no pacote enviado à Silterra.

3.2 Diferenças com relação aos Trabalhos anteriores

Este projeto se distingue do que havia sido proposto nos primeiros relatórios dessa disciplina em alguns aspectos, conforme pontuado pela introdução. Um deles é o design do chip, que agora inclui tanto ECC nas memórias quanto um pequeno microssistema de agregação e reportagem das falhas encontradas pelos mecanismos de ECC. A figura 3.1

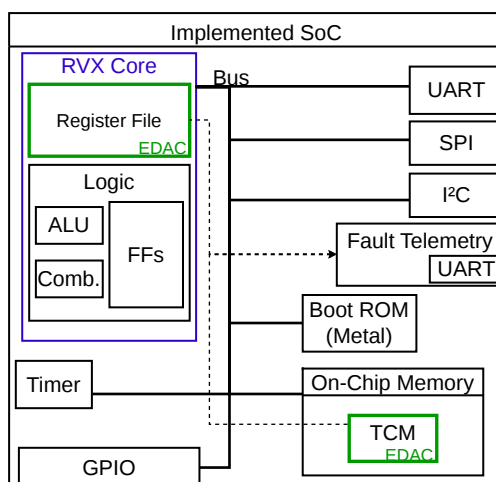


Figura 3.1 – Diagrama do SoC

Tabela 3.1 – Lista completa de PADs do design rvx

Função	Porta	PAD	Direção	Qtd.
Power	VDD	PVDD	in/out	4
Power	VSS	PVSS	in/out	4
Power	DVDD	PDVDD	in/out	4
Power	DVSS	PDVSS	in/out	4
Clock	clock	POSC4	in	1
Reset	reset_n	PIC	in	1
UART	uart_rx	PIC	in	1
UART	uart_tx	POCT2A	out	1
Tel. UART	rep_uart_rx	PIC	in	1
Tel. UART	rep_uart_tx	POCT2A	out	1
i ² c	i2c_sda_inout	PBCCT2A	in/out	1
i ² c	i2c_scl_inout	PBCCT2A	in/out	1
GPIO	gpio_inout	PBCCT2A	in/out	8
SPI	miso	PIC	in	1
SPI	sclk	POCT2A	out	1
SPI	mosi	POCT2A	out	1
SPI	cs	POCT2A	out	1
Total				36

3.3 Os resultados

A figura 3.3 é a imagem do chip depois de finalizado, com inserção de filler pads, ainda no ICC1. Já a figura 3.4 evidencia um corte próximo aos pads de VDD e VSS, na aresta norte do Die, evidenciando os trunks, o power ring e os stripes.

Com a inserção do ECC, a frequência alvo teve de ser reduzida de 50 para 38Mhz, em razão do caminho crítico do mar de flip-flops. Ao final, foi escolhida a frequência de 33.33MHz como alvo do tapeout.

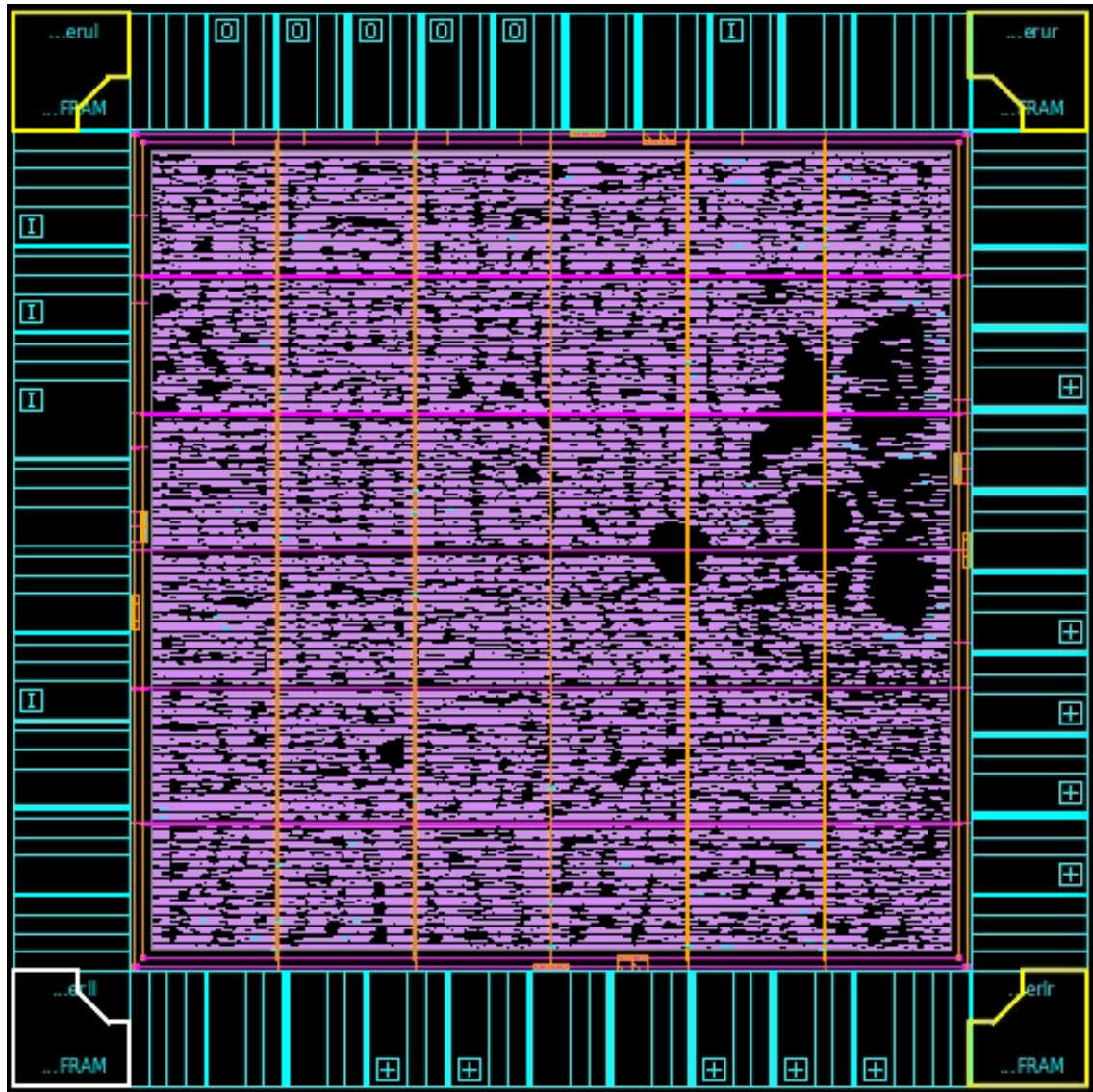


Figura 3.3 – Leiaute final

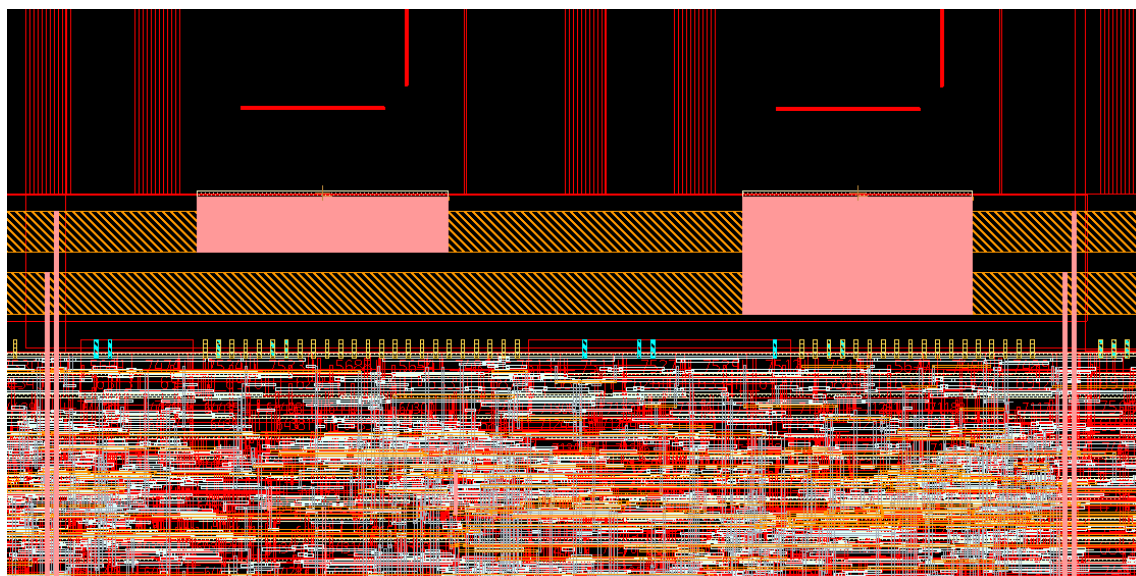


Figura 3.4 – Zoom na interface entre core e pads (capturada a partir do GDS aberto no custom Compiler)

3.4 Verificação e Signoff

Este design passou por verificação formal e simulação VCS entre todas grandes etapas de síntese. Ainda, passou pelo Calibre, que reportou zero erros de DRC no core e LVS limpo.

Ao longo do processo tivemos alguns erros de DRCs nos IPs de PAD providos, que foram waived pela Foundry. Além disso encontramos por vezes alguns poucos erros de DRC dentro do core, sobre espaçamento máximo de NWELL entre células, que solucionamos forçando um abbutment das mesmas através do desenho manual de um NWELL que as una. Mais detalhes serão providos no relatório seguinte.

4 CONCLUSÃO

A disciplina contribuiu com a fundamentação teórica necessária para conduzir de forma consciente cada etapa do fluxo, tanto na síntese lógica quanto na síntese física. O maior desafio esteve na interface entre o anel de PADs e o *core*, presente tanto em sky130 quanto em Silterra 180. A diferença é que, em Silterra, contamos com suporte para resolver os problemas encontrados, ao passo que o fluxo em sky130 ainda não foi explorado em sua totalidade no âmbito da universidade; o PDK disponível não permitiu levar o projeto até o final sem erros nos PADs.

O trabalho envolveu duas plataformas de EDA distintas: Cadence, no fluxo sky130, e Synopsys, no fluxo Silterra. No projeto em sky130, a memória foi reduzida ao mínimo, apenas para fins didáticos (128 *bytes*), enquanto na versão Silterra, destinada à produção, foi projetada com 1 KiB. É de nosso interesse aprender a utilizar o *memory compiler* em um eventual próximo *tapeout* com Silterra, recurso disponível para os nós tecnológicos de 110 e 130 nm.

Apesar de todos os desafios impostos, sobretudo em termos de tempo e de conhecimento técnico, alcançamos um *tapeout* completo e testado em Silterra 180, e conduzimos o fluxo em sky130 até o final, com o *core* livre de violações de DRC.

Repositório do projeto: <https://github.com/ulissesmaffa/rvx2gds>